1 Bibliothèque de communication et interface de contrôle

1.1 Contexte et motivation

1.1.1 Architecture matérielle et réseau

Le projet repose sur une flotte de trois bateaux autonomes *BlueBoat* de Blue Robotics (figure **??**), et d'une station de base terrestre. Chaque BlueBoat embarque un **Raspberry Pi** exécutant **BlueOS**, le système d'exploitation embarqué développé par Blue Robotics, ainsi qu'un **Pixhawk** (contrôleur de vol) qui exécute le firmware **ArduRover**. Le Pixhawk et le Raspberry Pi communiquent en local via une liaison USB/UART. La station de base et les trois bateaux sont reliés par un réseau **WiFi local** sur le sous-réseau 192.168.2.0/24 (figure **??**).

Figure 1: Un BlueBoat de Blue Robotics utilisé dans le projet.

1.1.2 Le protocole MAVLink

MAVLink (*Micro Air Vehicle Link*) est un protocole de communication léger, binaire et orienté messages, conçu pour les systèmes aériens et terrestres autonomes. Il est le standard de facto de l'écosystème ArduPilot/PX4.

MAVLink définit un ensemble de **messages standardisés** — par exemple `ATTITUDE` pour l'orientation (roll, pitch, yaw), `GLOBAL_POSITION_INT` pour la position GPS, `RC_CHANNELS_OVERRIDE` pour la commande des moteurs,

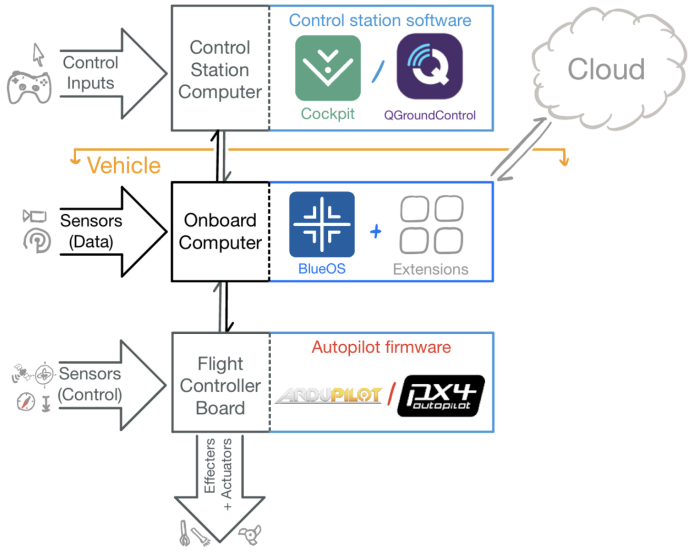

Figure 2: Pile logicielle embarquée sur un BlueBoat : le Pixhawk exécute ArduRover et communique avec le Raspberry Pi (BlueOS) via MAVLink série.

ou encore `COMMAND_LONG` pour l'envoi d'ordres tels que l'armement ou le changement de mode. Chaque message possède un identifiant numérique, une structure binaire fixe et est adressé par un couple (`system_id`, `component_id`) qui identifie le véhicule et le composant cible.

Dans l'usage traditionnel, MAVLink transite sur un **socket UDP ou TCP** : le logiciel de contrôle (Mission Planner, QGroundControl) s'abonne au flux de messages sur le port 14550 et doit maintenir une connexion active.

1.1.3 mavlink2rest : MAVLink accessible via HTTP

mavlink2rest est un serveur léger intégré à BlueOS qui fait le pont entre le flux MAVLink binaire du Pixhawk et une **API HTTP/REST**. Son fonctionnement est le suivant :

1. Il reçoit en continu les messages MAVLink du Pixhawk via la liaison série et les stocke en mémoire (toujours le message le plus récent de chaque type).

2. Il expose une API REST sur le port 6040 de chaque bateau, avec deux opérations :

- `GET /mavlink/vehicles/{sysid}/components/{compid}/messages/{MSG}` retourne en JSON le dernier message reçu de ce type ;
- `POST /mavlink` avec un payload JSON sérialise et injecte un message MAVLink dans le bus vers le Pixhawk.

Cette API est accessible depuis n'importe quel client HTTP sur le réseau local, sans configuration particulière. Une requête typique se présente sous cette forme :

Listing 1: Exemple de requête GET vers mavlink2rest pour lire la position GPS du bateau 2.

```python
import requests
r = requests.get(
    "http://192.168.2.202:6040"
    "/mavlink/vehicles/2/components/1/messages/GLOBAL_POSITION_INT"
)
data = r.json()
lat = data["message"]["lat"] / 1e7   # conversion de 1e-7 degres vers degres
lon = data["message"]["lon"] / 1e7
```

1.1.4 Pourquoi mavlink2rest plutôt que pymavlink ?

La bibliothèque Python officielle `pymavlink` permet d'ouvrir des connexions MAVLink bas niveau (UDP, TCP, série) et d'envoyer/recevoir des messages binaires. C'est l'approche classique. Cependant, dans notre contexte, elle présente plusieurs inconvénients majeurs.

Deux points sont particulièrement déterminants dans notre cas. D'abord, l'aspect **multi-clients** : pendant les tests, l'interface de supervision, les scripts de mission et les modules de logging tournent en parallèle sur la station de base, chacun devant accéder aux données des trois bateaux. Avec un socket MAVLink, le flux est consommé par le premier lecteur ; avec mavlink2rest, chaque composant interroge indépendamment le cache. Ensuite, l'aspect **intégration BlueOS** : sans modifier la configuration de BlueOS, aucun socket UDP MAVLink n'est publié vers l'extérieur, tandis que l'API REST est disponible sans intervention.

1.1.5 Besoin d'une bibliothèque commune

Les différents scripts du projet — tests unitaires, scripts de mission, interface graphique, boucle de localisation — partagent tous les mêmes opérations

fondamentales : lire le GPS, envoyer une commande moteur, lire le cap. Sans abstraction commune, chaque script réimplémenterait les requêtes HTTP, le parsing JSON MAVLink et les conversions de coordonnées, multipliant les sources d'erreurs et les duplications de code.

La bibliothèque `bblib` (*BlueBoat Library*) centralise cette logique sous une API orientée objet cohérente. Elle est testable indépendamment des bateaux physiques grâce au simulateur `fake_mavlink2rest.py` décrit à la section ??.

1.2 Architecture de `bblib.py`

1.2.1 Vue d'ensemble

`bblib.py` est organisé en cinq classes principales, dont les dépendances sont résumées ci-dessous :

```
MavlinkLink
  ↪ IMU
  ↪ GPS ⟶ geo_conversion
  ↪ MotorDriver
  ↪ Navigation(IMU, GPS, MotorDriver)
BlueBoatConfig ⟶ init_blueboat() ⟶ (MavlinkLink, IMU, GPS, MotorDriver, Nav
```

Les paramètres globaux (gain proportionnel K_p, période d'échantillonnage Δt, commande maximale) sont centralisés dans `utils/settings.py` pour éviter les constantes dispersées dans le code.

1.2.2 `MavlinkLink` — couche de communication REST

`MavlinkLink` est la seule classe à interagir directement avec le réseau. Elle encapsule une session `requests.Session` et fournit une interface haut niveau au reste de la bibliothèque.

Ses paramètres de construction sont l'adresse IP du bateau (`host`), le port mavlink2rest (`port`, par défaut 6040), les identifiants MAVLink (`sysid`, `compid`) et un délai d'attente (`timeout`). L'option `silent_errors` permet de supprimer les messages d'erreur dans les threads de monitoring qui tolèrent les coupures réseau passagères.

Les méthodes principales sont :

- `get_message(msg_name)` : requête `GET` vers l'endpoint mavlink2rest ; retourne le dictionnaire JSON du dernier message de ce type, ou `None` en cas d'échec.

- `post_message(message)` : requête `POST` avec le payload encapsulé dans un header MAVLink (`system_id` = 255, identifiant habituel d'une station sol GCS).
- `send_rc_override(ch1, ch3)` : construit et envoie un message `RC_CHANNELS_OVERRIDE` pour les canaux 1 (throttle) et 3 (steering).
- `arm_disarm(arm)` : envoi de `MAV_CMD_COMPONENT_ARM_DISARM`.
- `set_flight_mode(mode_id/mode_name)` : changement de mode ArduRover (`MANUAL`, `GUIDED`, `AUTO`...) via une table de correspondance nom $\leftrightarrow$ identifiant numérique.
- `get_flight_mode()` : lecture du mode actuel et de l'état armé/désarmé depuis le message `HEARTBEAT`.
- `get_battery_status()` : lecture de la tension, du courant et du pourcentage restant depuis `BATTERY_STATUS` et `SYS_STATUS`.

1.2.3 `IMU` — lecture d'attitude

La classe `IMU` lit l'orientation du bateau depuis le message MAVLink `ATTITUDE`, qui transmet les angles de roulis, tangage et lacet calculés par le filtre EKF (*Extended Kalman Filter*) embarqué dans ArduRover.

`get_euler_angles()` retourne le triplet (ϕ, θ, ψ) en radians. Un cache interne (`_last_euler`) conserve la dernière valeur valide, de sorte que la méthode ne retourne jamais `None` après la première lecture réussie, même lors de micro-coupures réseau.

`get_attitude_dict()` retourne un dictionnaire enrichi incluant `yaw_deg`, le cap normalisé dans $[0, 360]$, directement exploitable par la couche Navigation.

1.2.4 `GPS` — lecture de position

La classe `GPS` lit la position du bateau depuis deux sources MAVLink possibles, par ordre de priorité :

1. `GLOBAL_POSITION_INT` : position fusionnée par l'EKF (latitude, longitude, altitude, cap) ; valeurs entières en 10^{-7} degrés, à diviser par 10^7.
2. `GPS_RAW_INT` : données brutes du récepteur GNSS, utilisé en fallback si le premier message n'est pas disponible.

`get_gps()` retourne la position sous forme de tuple `(lat, lon)` en degrés décimaux et maintient un historique horodaté dans `gps_history`.

`get_coords()` retourne les coordonnées **cartésiennes locales** (x, y) en mètres, via la fonction `geo_conversion.conversion_spherique_cartesien()` qui effectue une projection sphérique autour d'un point de référence `POINT_BASE` défini dans `settings.py` (ici, le quai de Guerlédan). Cette représentation est celle utilisée par l'algorithme de localisation par intervalles.

`export_gpx(filename)` exporte la trace GPS au format GPX horodaté, compatible avec les logiciels SIG (QGIS, Google Maps) pour la visualisation des trajectoires.

1.2.5 `MotorDriver` — commande des moteurs

Le BlueBoat est propulsé par deux moteurs (gauche et droit) contrôlés par des signaux PWM (*Pulse-Width Modulation*) entre 1000 et 2000 µs, le neutre étant à 1500 µs. `MotorDriver` traduit des commandes de haut niveau en surcharges de canaux RC.

La conversion d'une paire de commandes moteur $(left, right) \in [-C_{max}, C_{max}]$ en signaux PWM se fait via la décomposition throttle/steering :

$$t = \frac{L+R}{2}, \qquad s = \frac{R-L}{2}$$

$$CH1 = 1500 - 500\,t, \qquad CH3 = 1500 + 500\,s$$

Les méthodes principales sont `send_cmd_motor(left, right)` pour un envoi instantané, `drive_lr(left, right, seconds, rate_hz)` pour une commande maintenue pendant une durée donnée (avec retour automatique au neutre), et `stop_motors()` pour interrompre immédiatement la propulsion. Pour la sécurité, le destructeur `__del__` coupe les moteurs et désarme le véhicule.

1.2.6 `Navigation` — navigation autonome

La classe `Navigation` réalise la boucle de guidage du bateau. Elle repose sur un **régulateur proportionnel** sur l'erreur de cap :

$$e = \psi_{cible} - \psi_{actuel} \quad (normalisedans[-180, 180])$$

$$correction = K_p{\cdot}e, \qquad \{\ moteurgauche = v_{base} - correctionmoteurdroit = v_{base} + correction$$

Plusieurs modes de navigation sont disponibles :

- `follow_heading(target_heading_deg, duration_s)` : suivi d'un cap fixe pendant une durée donnée, à vitesse constante.

- `go_to_gps(target_coords, distance)` : navigation vers un waypoint GPS avec une loi de vitesse de rapprochement $v = v_{base} \cdot \tanh(d/d_{ref})$, permettant un ralentissement progressif à l'approche.
- `go_to_position(target)` : commande vectorielle instantanée (1-shot) vers une position cartésienne locale. Cette méthode est utilisée pour l'intégration avec la boucle de l'algorithme de localisation.
- `return_home()` : séquence de waypoints prédéfinis vers la zone d'amarrage de Guerlédan.
- `stay_at(point)` : maintien de position par rappel doux.

1.2.7 `BlueBoatConfig` et `init_blueboat()`

Pour éviter de coder les adresses IP en dur dans les scripts, la configuration de la flotte est externalisée dans un fichier JSON (`boat_control_config.json`) :

Listing 2: Exemple de fichier de configuration de la flotte (`boat_control_config.json`).

```
[
  {"boat_id": 1, "host": "192.168.2.201", "port": 6040, "sysid": 1},
  {"boat_id": 2, "host": "192.168.2.202", "port": 6040, "sysid": 2},
  {"boat_id": 3, "host": "192.168.2.203", "port": 6040, "sysid": 3}
]
```

La classe `BlueBoatConfig` charge ce fichier avec des valeurs par défaut en cas d'absence. `init_from_config(boat_id)` instancie en une seule ligne l'ensemble des composants d'un bateau (`MavlinkLink`, `IMU`, `GPS`, `MotorDriver`, `Navigation`). `init_all()` parcourt tous les bateaux configurés et retourne un dictionnaire indexé par `boat_id`, utilisé par les scripts de mission multi-bateaux.

1.3 Interface graphique de supervision — `boat_control_gui.py`

1.3.1 Motivation

Pour les essais terrain, l'équipe avait besoin d'un outil permettant de surveiller l'état des bateaux en temps réel et d'envoyer des commandes simples sans passer par la ligne de commande. L'interface `boat_control_gui.py` répond

à ce besoin. Elle est entièrement construite sur `bblib`, ce qui valide en pratique que la bibliothèque couvre l'ensemble des cas d'usage (figure **??**).

images/boat_control_python.png

Figure 3: Interface graphique de supervision et de contrôle des trois bateaux (`boat_control_gui.py`).

1.3.2 `HeartbeatManager` — maintien de la connexion GCS

Le protocole MAVLink exige qu'une station sol (GCS) envoie régulièrement un message `HEARTBEAT` pour signaler sa présence. Si BlueOS ne reçoit pas de heartbeat depuis la station pendant plusieurs secondes, il peut désarmer le véhicule ou couper la connexion. Le `HeartbeatManager` lance un thread dédié qui envoie ce message à 1 Hz vers chacun des bateaux configurés, en parallèle de toute autre activité de l'interface.

1.3.3 `BoatMonitor` — acquisition des données

Un objet `BoatMonitor` est instancié pour chaque bateau. Il encapsule les composants `bblib` du bateau correspondant et lance un thread de polling à environ 2 Hz qui met à jour en continu les données affichées : position GPS, cap, tension et pourcentage de batterie, mode ArduRover et état armé/désarmé. La détection du nombre de cellules de la batterie (3S ou 4S) est automatique à partir de la tension mesurée, ce qui permet de calculer un pourcentage fiable sans configuration manuelle.

1.3.4 Interface Tkinter

L'interface est organisée en panneaux, un par bateau, affichant :

- **GPS** : latitude et longitude en degrés décimaux ;
- **Cap** : heading en degrés, mis à jour depuis l'`IMU` ;
- **Batterie** : tension en volts et pourcentage estimé ;
- **Mode** : mode ArduRover actif et indicateur armé/désarmé.

Des boutons permettent d'**armer**, de **désarmer** et de lancer un **retour maison** (exécuté dans un thread séparé via `Navigation.return_home()` pour ne pas bloquer l'interface). La configuration est chargée dynamiquement depuis `boat_control_config.json` : il suffit de modifier les adresses IP dans ce fichier pour adapter l'interface à un autre réseau. Enfin, les connexions indisponibles sont gérées explicitement : un bateau hors réseau affiche des valeurs dégradées sans bloquer les autres panneaux.

1.4 Simulateur `fake_mavlink2rest.py`

Pour permettre le développement et les tests sans accès physique aux bateaux, un serveur HTTP simulant l'API mavlink2rest a été développé dans `fake_mavlink2rest.py`. Il démarrage un serveur HTTP léger par port (un par bateau simulé), chacun générant des données synthétiques :

- Position GPS animée autour d'un point de base avec bruit gaussien ;
- Attitude (roll, pitch, yaw) avec bruit et dérive ;
- Tension de batterie décroissante au fil du temps ;
- Gestion des commandes `POST` : armement, changement de mode, RC override.

Ce simulateur a été utilisé lors de la semaine de formation pour tester rapidement les boucles de commande, valider `bblib` et `boat_control_gui.py`, et vérifier les scripts de mission multi-bateaux, sans mobiliser le matériel.

1.5 Tests de la première semaine de formation

1.5.1 `test_bblib.py` — tests CLI de la bibliothèque

Le script `test_bblib.py` fournit une interface en ligne de commande pour tester chaque fonctionnalité de `bblib` de manière isolée, en se connectant soit au simulateur soit à un bateau réel. Le tableau **??** résume les commandes disponibles.

1.5.2 Autres scripts de test

Plusieurs scripts complémentaires ont été développés durant la semaine de formation :

- `test_mavlinkrest.py` : tests bas niveau de l'interface REST — vérification des codes HTTP, structure des réponses JSON, latences de requêtes GET et POST.
- `test_set_mode_manual.py` : validation de la commande de changement de mode ArduRover.
- `test_load_config.py` : vérification du parsing JSON et des valeurs par défaut de `BlueBoatConfig`.
- `test_multiple_boat.py` : initialisation simultanée des trois bateaux et polling en parallèle pour valider l'absence de blocage.
- `test_formation_triangle.py` : premier essai de formation en triangle (voir figure **??**), utilisant `go_to_position()` sur les trois bateaux coordonnés depuis la station de base.

1.5.3 Résultats et observations

Les tests ont permis de valider le bon fonctionnement de l'ensemble de la chaîne de communication. Quelques points de vigilance ont été identifiés :

- **Latence REST** : chaque requête `GET` vers mavlink2rest prend typiquement 50 à 100 ms sur le réseau WiFi local. Cela impose une fréquence de contrôle maximale d'environ 10 Hz pour la boucle de guidage, ce qui s'est avéré suffisant pour la dynamique lente des bateaux.

images/test_formation.png

Figure 4: Trace GPS d'un test de formation en triangle avec les trois bateaux.

- **Heartbeat obligatoire** : sans l'envoi régulier du `HEARTBEAT` GCS, BlueOS refusait les commandes RC override. Le `HeartbeatManager` a été développé en réponse directe à ce comportement observé lors des premiers tests.
- **Robustesse réseau** : les coupures WiFi passagères sont gérées par l'option `silent_errors` de `MavlinkLink` et le cache interne de l'`IMU`, qui évitent les exceptions non contrôlées dans les threads de monitoring.
- **Précision GPS** : la précision observée est de l'ordre de 1 à 3 mètres, cohérente avec du GPS standard sans correction différentielle.

1.6 Bilan

La bibliothèque `bblib` fournit une abstraction complète de la communication MAVLink/REST pour les BlueBoat. Son API orientée objet a permis de développer rapidement et de façon fiable l'interface de supervision, les scripts de mission multi-bateaux et la boucle d'intégration avec l'algorithme de localisation par intervalles décrit à la section **??**.

Le choix de mavlink2rest plutôt que pymavlink s'est avéré particulièrement judicieux dans ce contexte : il supprime toute gestion de socket, permet un accès multi-clients naturel et exploite l'API déjà exposée par BlueOS. La

contrepartie — une latence légèrement plus élevée qu'un accès UDP direct — reste négligeable au regard de la dynamique des bateaux de surface.

Table 1: Comparaison de `pymavlink` et `mavlink2rest` dans le contexte du projet.

Critère	`pymavlink` **(socket brut)**	`mavlink2rest` **(HTTP/JSON)**
Couche réseau	Socket UDP/TCP à maintenir, gestion manuelle de la reconnexion	Simple requête HTTP sans état
Accès simultané	Un seul consommateur du flux (le socket draine les messages)	Plusieurs clients lisent le cache indépendamment
Intégration BlueOS	BlueOS n'expose pas de socket UDP MAVLink par défaut ; configuration réseau nécessaire	Nativement disponible sur le port 6040, sans configuration
Débogage	Messages binaires, outils spécialisés requis (Wireshark, logs pymavlink)	Testable directement avec `curl` ou un navigateur
Dépendances	Bibliothèque lourde avec bindings C	Seulement `requests` (bibliothèque standard Python)
Multi-bateaux	N sockets distincts, N threads de réception permanents	N instances légères d'`MavlinkLink`, sans thread réseau dédié

Table 2: Commandes du script de test `test_bblib.py`.

Commande	**Fonctionnalité testée**
`imu`	Lecture du message `ATTITUDE` → roll / pitch / yaw en radians et degrés
`gps`	Lecture de `GLOBAL_POSITION_INT` ou `GPS_RAW_INT` → lat/lon/alt/cap
`arm`	Armement du véhicule via `MAV_CMD_COMPONENT_ARM_DISARM`
`disarm`	Désarmement du véhicule
`battery`	Lecture de la tension et du pourcentage de batterie
`neutral`	Envoi d'un RC override neutre (CH1=CH3=1500 µs)
`drive`	Commande moteurs pendant N secondes à gauche/droite spécifiés
`heading`	Suivi de cap avec le régulateur proportionnel
`goto`	Navigation vers un waypoint GPS
`config`	Chargement et affichage de `boat_control_config.json`